Day 2 Take-Home – Model tiers, embeddings and verification

Repository: `automation-to-agentic-spectrum` (local clone in `training/week1/day2`). All data is synthetic. No `.env`, API key or real data is included. Raw output of every run is in `outputs/day2_runs/`; the reports are in `outputs/`.

Before you start

Check	Result
In-lab tier comparison	`outputs/tier_comparison_Q1.md` (and Q2, Q3, custom)
Homework pack	`outputs/homework_comparison.md` generated
Unit tests	55 OK on a fresh clone (after adding the missing `.env.example`); **63 OK** after my changes (3 skipped, 1 expected failure, explained in Part C)

Part A – Tier study

Claude Code CLI, one call per tier, no tools, one turn. Correctness is my own judgement against the reference answers, not the automatic number check. Word counts include markdown labels.

Query	Tier	Correct? (0-3)	Seconds	Cost ($)	Followed length limit?	Notes
Q1	opus	3	18.9	1.3117	Yes (122 / 150)	All three right; also noted the 90-day notice still applies
Q1	sonnet	3	5.9	0.6626	Yes (118 / 150)	All three right, cites each clause
Q1	haiku	**2**	4.3	0.1668	**No (155 / 150)**	**Opens with a wrong figure ("15%… 1,860.00"), then works to 620.00.** Auto-check still said 2/2
Q2	opus	3	19.8	1.3222	Yes (2 sentences)	Accurate; noted "no currency is stated"
Q2	sonnet	3	2.8	0.0319	Yes (2 sentences)	Accurate and tight
Q2	haiku	3	4.1	0.2057	Yes (2 sentences)	Accurate; most detail (5% per step)
Q3	opus	3	5.8	0.0622	**No (92 / 80)**	Good risks and alternative
Q3	sonnet	3	4.2	0.0311	Yes (76 / 80)	Good risks and alternative
Q3	haiku	3	4.4	0.1252	**No (88 / 80)**	Good risks and alternative

Reading the cost column. The CLI cost is dominated by Claude Code's own system prompt, not by the question. Input tokens show as 2 because that prompt is cached. The first call per tier that finds the cache cold pays for writing it (Opus Q1/Q2 about $1.32), and later calls only read it (Opus Q3 $0.06). "Served by" also lists Haiku on every tier, because the CLI uses it for a small background task. So these costs compare *CLI sessions*, not models. For a model-to-model cost comparison I would use the API directly and the list prices (per million tokens, in/out: Opus 5.5 $4 / $20, Sonnet 5.5 $2 / $10, Haiku 5.5 $0.10 / $0.50).

Recommendation.

- **Reasoning and arithmetic on documents (Q1): Sonnet.** It was correct, within the limit, and 3× faster than Opus. Haiku is not safe unchecked here: its answer contained a wrong figure next to the right one, which a quick reader or a keyword check would pass.

- **Summarisation (Q2): Haiku, with spot checks.** All three tiers were accurate. At list price Haiku is about 20× cheaper per token than Sonnet, which matters at volume. Summaries are low risk if the source stays one click away.
- **Judgement / policy advice (Q3): Sonnet.** All three gave sound advice, but only Sonnet kept to 80 words. For text a person reads and acts on, following instructions counts.
- **Opus:** no measurable quality gain on these three tasks, and the slowest. I would keep it for long, multi-step or ambiguous work and confirm the benefit on that work before paying for it.
- **Caveat:** one run per cell. Outputs vary between runs, so a production choice needs a small eval set of 20–50 labelled cases per task type, not one example.

Part B – My own query across six tools

Query (`solutions/day2_models_and_embeddings/data/my_query.txt`): a fictional credit-card terms extract (annual fee, non-sterling fee, late fee, cash advance, disputes), about 150 words, with three questions that have known answers:

1. Spend 8,400.00, waiver needs 9,000.00 → annual fee **charged, 180.00**
2. Cash advance 60.00 → 3% = 1.80, but the **minimum 3.00** applies (the deliberate trap)
3. Purchase of 400.00 in euros → 2.75% = **11.00**

`redact_pii.py --file` found nothing (`outputs/day2_runs/partB_redaction_check.txt`).

Tool	Q1	Q2 (trap)	Q3	Under 120 words	Seconds	Notes
Claude Opus (CLI)	✓ 180.00	✓ 3.00	✓ 11.00	Yes (89)	4.8	Showed the minimum-fee working
Claude Sonnet (CLI)	✓	✓	✓	Yes (93)	3.0	Flagged that the terms don't say whether 400.00 is in EUR or its GBP value
Claude Haiku (CLI)	✓	✓	✓	Yes (76)	3.6	Same ambiguity noted
Lab Q1 prompt (from `homework_pack.py`) as a fresh conversation. The identical text, with no system prompt and no						
history. Claude was run through the Anthropic API on the models the Claude.ai app offers. ChatGPT and Gemini could not						
be run from this machine (no browser access to the apps, no API key), so their columns are empty rather than guessed.						

Tool	Credit 620.00	Charge 24,800.00	Quoted 11.4	Under 150 words	Seconds	Added beyond the clauses
Claude Sonnet 5.5 (Claude.ai default model, via API)	✓	✓	✓ verbatim	✓ 142	3.7	Labelled caveats: notice-period fees not addressed; 11.4 covers personal data only
Claude Opus 5.5 (via API)	✓	✓	✓ verbatim	✓ 126	5.4	Notice-period caveat; "no currency is stated"
ChatGPT (web)	not run					No access from this machine
Gemini (web)	not run					No access from this machine

Settings check: done from the vendors' help pages, to be confirmed in your account (full table in `outputs/homework_comparison.md`). * ChatGPT: *Settings → Data controls → Improve the model for everyone*, on by default for Free/Plus/Pro. * Claude.ai: *Settings → Privacy → Help improve Claude*, on unless you opted out. * Gemini: *Settings & help → Activity → Keep Activity*, on by default; human reviewers see a subset of chats.

Business and enterprise plans of all three do not train on your content by default.

Reflection. Across my own query and the lab's Q1 prompt, every Claude run reached the right answers, including the minimum-fee trap and the "full steps only" credit rule. I would still not trust any answer *unchecked* when money is involved. The one I would come closest to trusting is Opus or Sonnet on short, well-defined clauses, because both showed their working and quoted the clause, so checking takes seconds. The one I would check first is Haiku. In the CLI run of Q1 it put a wrong figure (1,860.00) next to the right one (620.00), the automatic number check passed it, and only reading every line caught it. Within any answer I would check derived figures first (caps, minimums, thresholds), because that is where arithmetic and the reading of conditions meet. The best answers also said what the source does *not* cover: notice-period fees, the currency of the 400.00, whether 11.4 covers non-personal data. That is the behaviour I want, because it hands the reviewer the open questions instead of hiding them. Speed and length were close enough not to matter here: 3–6 seconds, all inside the limit, though Sonnet's 142 of 150 words shows that limits are met with little margin. A fair cross-vendor verdict still needs ChatGPT and Gemini run under the same rules, which was not possible from this machine, so I make no claim about them. The data-handling side differs more than the answers do. All three consumer apps may use chats for training by default, while the organisation's API key and enterprise plans do not, so the choice of account matters more than the choice of model. The comparison taught me to pick a tool per task *and* per check: the model that shows its working is the one I can verify, and the account type decides what I am allowed to paste at all.

Part C – Embeddings experiment

New topic: `finance_expenses`, 8 synthetic sentences (taxi receipts, hotel limit, mileage claim, itemised meal receipt, corporate card reconciliation, travel agency, sign-off while the manager is on holiday, paying back personal items).

Fixed on the way: `plot_map()` had a 4-colour palette zipped against the topics, so a **fifth topic was silently dropped from the map**. The palette now has 8 colours and the function raises an error if there are more topics than colours. I also added a `--query` option so extra queries can be tested.

Toy embedder (`outputs/day2_runs/partC_toy.txt`, map `partC_embedding_map_toy.png`):

- The four original topics still cluster tightly. The expense sentences **do not form a cluster**: they scatter between the refunds, and one lands inside the HR-leave cluster.
- "How much vacation am I entitled to?" now returns **an expense sentence first** ("Who signs off expense claims when my line manager is on holiday?", 0.98), above every HR-leave sentence. *This is the query where the toy embedder fails.* It counts words from six fixed groups. "holiday" and "manager" are both in its *leave* group, and it has no *expenses* group at all. So that sentence looks like pure leave, and the query matches it on two words rather than on meaning.
- The expense queries ("How do I get reimbursed for a train ticket from a work trip?") do land on expense sentences, but only by accident. None of their words is in a concept group ("reimbursed" ≠ "reimburse"), so the result comes from the checksum "nudges" for unknown words. That is not a success I would trust.

Real model: *not run on this machine.* `fastembed` installs, but `onnxruntime` cannot load (DLL error). The Microsoft Visual C++ x64 runtime is missing, and installing it needs an admin install. A real 384-dimension model learns that "vacation", "annual leave" and "holiday" mean the same, and that "expense claim" is its own concept. It should put the vacation query on HR leave and the expense queries on expenses, but that remains to be confirmed once the runtime is installed (`embeddings_map.py` without `--toy`).

Tests (`tests/test_day2.py`): * The vacation case is now a documented `expectedFailure` for the toy embedder, with the reason in its docstring. It is kept visible on purpose, not deleted. * A new `RealEmbeddings` class checks that a synonym query finds `finance_expenses` and that the vacation query finds `hr_leave`. It is skipped automatically where fastembed cannot load. * With the toy embedder this cannot be tested meaningfully. It has no word group for expenses, so any "pass" would come from checksum noise and could break when an unrelated word is added.

What "distance" on the map means, and why the axes have no fixed meaning. Two points close together on the map had embedding vectors pointing in similar directions, so the model treats those sentences as similar in meaning; far apart means different. The map is a flattening, though. PCA squeezes 6 (or 384) dimensions into the 2 directions with the most spread, so some closeness is lost and two points can look near on paper while being further apart in the full space. The axes are just those two directions of greatest variation in *this* set of sentences. They are blends of all the original dimensions, not "money" or "delivery". Add a topic or change one sentence and the directions change: the map can rotate, flip or stretch, so only the relative positions mean anything.

Part D – Extend the verification pipeline

New item: `late_interest_percent_per_month` = **1.5** (clause 4.2). It was added to `EXTRACT_SYSTEM`, `ground_truth()`, `NUMBER_IN_QUOTE` and the offline reply (correct there, so the demo still shows only its two original deliberate errors).

Bug found and fixed: `_number_variants()` also produced whole-number forms of every value, so 1.5 became `"2"`. The "number is in the quote" check would then pass for any quote containing a 2, for example just "4.2 Late payment." Whole-number forms are now only produced for whole numbers. A test covers this.

Tests added: new item correct → LOW; new item with an invented quote → HIGH; the rounding bug; the coverage check (below). Day 2 verification tests: 10, all passing.

`--offline-demo` (`outputs/day2_runs/partD_offline_demo.txt`): still **HIGH**. The invented 25% cap quote and the 1,240 credit are caught; the new item passes.

Live (Sonnet 5.5, `outputs/day2_runs/partD_live.txt`), run 1 of 3:

```
item                              model  expected  quote exists  number in quote  value ok
monthly_fee                     12400.0     12400  yes           yes              yes
credit_percent_per_step             5         5  yes           yes              yes
credit_cap_percent                 20        20  yes           yes              yes
credit_for_98_6                 620.0       620  yes           n/a              yes
termination_charge_8m           24800.0     24800  yes           n/a              yes
late_interest_percent_per_month     1.5       1.5  yes           yes              yes
Coverage: clauses with figures not cited by any quote: none
RISK TIER: LOW
```

Audit line: `{"time": "2026-10-29T04:30:45+00:00", "source": "claude-sonnet-5-5", "tier": "LOW", "failed_items": [], "uncited_clauses": []}`

Of the first 4 live runs, 2 returned valid JSON (LOW) and 2 did not. On those the script stopped without acting, which is correct, **but also without writing an audit line**, so the worst outcome left no trace. It now logs an unusable answer as HIGH and prints the model's stop reason. That showed the cause: `stop_reason: max_tokens` at exactly 800 output tokens. The reply was cut off mid-JSON, because the model's thinking counts toward the cap. Raising `max_tokens` for the extraction call to 2,000 fixed it: **3/3 further live runs LOW** (492–765 output tokens, `partD_live_after_fix.txt`).

If every check passed but the model had silently left out clause 9.2 altogether, what would catch it? Nothing in the original pipeline. It only checks the items it asks for, so an item that never comes back is invisible if the

schema doesn't ask for it, and a clause the schema forgot is never looked at. **Proposed and implemented: a coverage check.** `uncited_clauses()` splits the contract into its numbered clauses and lists every clause that contains a figure but that no extracted quote comes from. Any uncited clause raises the tier to at least **MEDIUM** ("Clause(s) 9.2 contain figures but were not covered. A person must review them.") and is recorded in the audit log. Clause 11.4 has no figure and is correctly ignored. Tested: dropping the termination item gives `["9.2"]` and MEDIUM.

Part E – Approved-tools guideline

See `docs/Approved_Tools_Guideline_DRAFT.md` (one page).